

TLS hardening build and support manual

Windows Schannel audit hardening and sslscan extension

This manual explains setup, operation, PowerShell code, recovery, and the build plan for an optional sslscan extension.

The script starts with a local audit. It changes Windows settings only after the operator gives the final confirmation.

Document item	Value
Owner	RuleRivet
Document revision	1.1
Issue date	9 September 2026
Script	Invoke-TLSHardening.ps1 version 1.0.1
Audience	Windows administrators and technical support personnel
Status	Technical support manual and extension build plan

Document scope

Sections 1 to 6 describe version 1.0.1. Sections 7 to 10 specify the proposed extension and release tests.

The proposed extension is not part of version 1.0.1.

CAUTION A TLS change can stop remote access or an application. Confirm console access before you select hardening.

The local audit does not prove which TLS settings a network service accepts. Application tests and network scans supply different evidence.

Contents

Use this page to find the setup, operation, support, and build information.

Section	Subject	Page
1	Prepare the computer and files	3
2	Run the audit and hardening	4
3	Understand protocol settings	5
4	Understand the PowerShell functions	6
5	Recover the previous configuration	7
6	Find and correct faults	8
7	Build the next release	9
8	Set up sslscan	10
9	Add the scan stage	12
10	Accept the release and retain evidence	14
11	Terms and references	15

How to use this manual

Start with section 1 before the first run. Use section 2 for an approved change.

Use sections 3 and 4 to understand the results and the code. Use sections 5 and 6 for support and recovery.

Use sections 7 to 10 to build and test the sslscan extension.

1 Prepare the computer and files

Prerequisites

Use a Windows release that your organisation supports. Use 64-bit Windows PowerShell 5.1. Hardening needs local administrator rights and the Windows TLS module.

The main script needs no compiler, Python installation, or paid service. Its backup function calls the built-in reg.exe program.

1. Copy the package to C:\Admin\TLS-Hardening.
2. Extract the package.
3. Keep Invoke-TLSHardening.ps1 and README.md together.
4. Open Windows PowerShell.
5. Go to the folder that contains Invoke-TLSHardening.ps1.
6. Run the checks below.

```
Set-Location 'C:\Admin\TLS-Hardening\Interactive-TLS-Hardening'  
$PSVersionTable.PSVersion  
[Environment]::Is64BitProcess  
Get-Command Get-TlsCipherSuite, Disable-TlsCipherSuite  
Get-ExecutionPolicy -List  
Get-FileHash .\Invoke-TLSHardening.ps1 -Algorithm SHA256  
Get-AuthenticodeSignature .\Invoke-TLSHardening.ps1
```

The version must be 5.1 or later. The process check must return True. Both TLS commands must exist for hardening.

A hash records file identity. It proves origin only when it matches a trusted value. An unsigned file can return NotSigned.

Script execution

1. Inspect the script before execution.
2. Use the script signing or application approval process for your organisation.
3. If a download block remains, remove it only after file approval.
4. If central policy blocks execution, refer the failure to the policy owner.

```
Unblock-File -LiteralPath .\Invoke-TLSHardening.ps1
```

Unblock-File removes the download marker. It does not override AppLocker, WDAC, or a central execution policy.

Before a change

1. Record the computer name, Windows build, change reference, and application owner.
2. Agree the maintenance period and the recovery method.

3. Do the recovery test in section 5 on a representative test computer.
4. Record normal application operation before the change.

2 Run the audit and hardening

Report only

1. Run the command below.
2. At the mode prompt, enter R.
3. Keep a copy of the displayed results.

```
.\Invoke-TLSHardening.ps1
```

An empty response selects report only. Every response except H selects report only. The script converts the mode response to uppercase.

Version 1.0.1 displays results in the console. It does not create an HTML, CSV, or XML report.

For a durable console record, start a transcript before the run. Use the approved evidence folder.

```
New-Item -ItemType Directory -Path 'C:\Admin\TLS-Evidence' -Force
$stamp = Get-Date -Format 'yyyyMMdd-HH:mm:ss'
Start-Transcript -Path "C:\Admin\TLS-Evidence\audit-$stamp.txt"
.\Invoke-TLSHardening.ps1
Stop-Transcript
```

The transcript writes a file. It does not change TLS settings. PowerShell ISE can run the script with F5.

Hardening

CAUTION Both client and server settings will change. Confirm that outgoing connections can also use the selected protocols.

1. Open Windows PowerShell with Run as administrator.
2. Run the script from its folder.
3. At the mode prompt, enter H.
4. At the TLS 1.2 prompt, select Y if the approved plan enables TLS 1.2.
5. If the TLS 1.3 prompt appears, select the value in the approved plan.
6. Read the proposed changes.
7. To apply the changes, type HARDEN exactly.
8. Record the backup folder and all result messages.
9. Restart Windows during the approved maintenance period.
10. Run report-only mode again.
11. Do the application and network tests in section 10.

N leaves the selected modern protocol unchanged. It does not disable that protocol. Other final confirmation text cancels the change.

The script never restarts Windows. A successful result confirms command completion, not application compatibility. Errors can leave a partial change.

3 Understand protocol settings

Local state and network state

Schannel is the Windows TLS provider. A program can use a different TLS provider. A proxy can terminate TLS before traffic gets to Windows.

The script reads protocol values from this registry location:

```
HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\  
SCHANNEL\Protocols
```

The displayed path above wraps for readability. Each protocol has separate Client and Server subkeys.

Displayed state	Meaning in this script
OS default	Neither value was read successfully. Windows defaults can apply; a read error can also cause this result.
Explicitly disabled	Enabled equals 0.
Explicitly enabled	Enabled is nonzero; DisabledByDefault is absent or 0.
Enabled; off by default	Enabled and DisabledByDefault are both nonzero.
Disabled by default	Enabled is absent; DisabledByDefault is nonzero.
Available by default	Enabled is absent; DisabledByDefault equals 0.

Get-RegistryValue catches all read errors. Thus, OS default is not reliable evidence when registry access fails.

Changes made by Set-ProtocolState

Selection	Enabled DWORD	DisabledByDefault DWORD
Disable protocol	0	1
Enable protocol	1	0

The script disables SSL 2.0, SSL 3.0, TLS 1.0, and TLS 1.1 for both roles. SSL 1.0 is not a configurable Schannel protocol.

OS_Support is a build-based estimate. It is not a handshake result. A registry key cannot add support to Windows.

The TLS 1.3 threshold is build 20348 for servers and build 22000 for clients. Microsoft identifies Server 2022 and Windows 11 as the initial supported releases. [R2]

If the CIM query fails, the fallback marks the computer as a client. Verify the OS before you rely on that result.

4 Understand the PowerShell functions

Function	Purpose and important behaviour
Write-Section	Displays a console heading. It does not return audit data.
Get-RegistryValue	Returns Exists and Value. Every exception becomes Exists = False.
Get-ProtocolSetting	Converts two registry values into the states in section 3.
Test-IsAdministrator	Checks the current Windows token. It does not request elevation.
Read-YesNo	Accepts Y, YES, N, or NO. Other responses cause another prompt.
Get-LegacyCipherReason	Checks a cipher name against the configured text patterns.
Set-ProtocolState	Creates the protocol subkeys and writes two DWORD values for each role.
Export-RegistryBackup	Runs reg.exe export. A nonzero exit code stops the change before TLS writes.

Cipher audit and the version 1 point 0 point 1 correction

```
$enabledCipherSuites = @(Get-TlsCipherSuite)
foreach ($nameValue in @($nameProperty.Value)) {
    $cipherName = [Convert]::ToString($nameValue)
    # The script checks each nonempty name separately.
}
```

This excerpt shows the conversion inside the audit loop. The full script first checks for a Name property.

@(...) produces an array. This keeps Count reliable for zero, one, or several results. Each name becomes one string before classification.

This addresses the reported Name conversion error. Disable-TlsCipherSuite receives one suite name for each call. [R4, R5]

The patterns match NULL, RC2, RC4, DES, 3DES, EXPORT, MD5, and CBC. CBC removal is a strict policy choice.

A CBC match does not prove that a service has an exploitable fault. The rules do not check every possible cryptographic weakness.

Control flow

The main block checks Windows, reads the OS, builds protocol rows, and audits ciphers. It then requests the operating mode.

Hardening stops without administrator rights, usable cipher commands, a successful cipher audit, or when the checked cipher policy exists.

After confirmation, both registry exports must succeed. Protocol writes run first. Cipher changes follow, including after a protocol failure.

StrictMode 2.0 catches some invalid variable and property access. ErrorActionPreference Stop makes many command errors enter a catch block.

5 Recover the previous configuration

Backup contents and limits

The script creates this folder after final confirmation:

```
%ProgramData%\TLS-Hardening-Backups\yyyyMMdd-HHmms\
  SCHANNEL.reg
  CipherSuites.reg
```

SCHANNEL.reg contains the Schannel branch. CipherSuites.reg contains the local SSL configuration branch named 00010002.

A registry import merges values. It does not remove values that the script created after the export. Imports alone are not exact recovery.

The package has no restore script. The operator needs a tested recovery procedure before production hardening.

Prepare the recovery record before hardening

1. Record whether each Client and Server key exists for all six protocols.
2. For each key, record Enabled and DisabledByDefault existence, type, and value.
3. Save the full ordered cipher list with the command below.
4. Confirm that the system recovery method is valid for this server role.
5. Keep the record with the change reference and computer identity.

```
Get-TlsCipherSuite | Export-Clixml `
  -LiteralPath 'C:\Admin\TLS-Evidence\ciphers-before.xml'
```

For registry capture, use Registry Editor or an approved inventory tool. An absent value must be recorded as absent, not zero.

Recover after a failed change

CAUTION Do not delete the complete Schannel branch. That action can remove unrelated settings.

1. Open the console access path if remote access fails.
2. Stop further changes on the computer.
3. Confirm that the backup belongs to this computer and this change.
4. Compare current values with the recovery record.
5. Restore each changed protocol value to its previous type and value.
6. Remove a changed value only when the record proves that it was previously absent.
7. Restore the saved local cipher configuration through the approved recovery procedure.
8. Check the cipher order against ciphers-before.xml.
9. Restart Windows during the recovery period.
10. Do the application and network tests again.

Remove a newly created key only if it is empty and its previous absence is recorded. Preserve unrelated changes.

If the previous state is unknown, use the approved system recovery method. Do not guess the former protocol configuration.

6 Find and correct faults

Symptom	Action
Name conversion failure	Confirm version 1.0.1. Check that the loop converts each Name value separately. Keep the full error text.
TLS commands missing	Use 64-bit Windows PowerShell. Check the Windows TLS module. Do not force the hardening path.
Hardening needs administrator rights	Reopen Windows PowerShell with Run as administrator. Run the audit again.
Cipher policy detected	Ask the policy owner to change SSL Cipher Suite Order. Do not delete the policy to pass the check.

Symptom	Action
Registry backup failed	Check the destination, access rights, free space, registry branch, and reg.exe approval. Do not continue without both exports.
One or more change errors	Treat the state as partial. Keep the failure list. Use the tested recovery procedure or an approved repair.
Old TLS remains visible	Check restart status, target IP, SNI name, proxy, TLS provider, and policy. Scan each TLS termination point.
OS default seems incorrect	Check registry access independently. Get-RegistryValue can hide an access failure.
Application fails after restart	Use console access. Check the application dependency and both TLS roles. Recover the previous state if necessary.
No report file exists	Version 1.0.1 has console output only. Use a transcript for a durable record.

Useful support data

```
Get-CimInstance Win32_OperatingSystem |
    Select-Object Caption, BuildNumber, ProductType
Get-Module -ListAvailable TLS
Get-TlsCipherSuite | Select-Object Name
Get-NetTCPConnection -State Listen |
    Select-Object LocalAddress, LocalPort, OwningProcess
```

A listening port does not prove TLS support. The process ID helps the operator find the application owner.

Exit code 0 can mean report completion, cancellation, or successful configuration. Exit code 1 can mean a blocked or failed change.

The exit code alone cannot establish compliance. Keep the selected mode and the console result with it.

7 Build the next release

The proposed release keeps one interactive PowerShell entry point. sslscan is an optional external program. It never decides whether to change Windows.

Build sequence

1. Create a development copy of version 1.0.1.
2. Replace ambiguous registry reads with Found, Absent, and ReadError results.
3. Stop hardening when a required read returns ReadError.

4. Add a per-value snapshot with key existence, value existence, type, and data.
5. Add computer identity, UTC time, script hash, and cipher order to the snapshot.
6. Save the snapshot in a unique run folder before each change.
7. Add a restore function that reverses only recorded changes.
8. Prove restoration of absent values in a test environment.
9. Add the optional target list and scan stage from sections 8 and 9.
10. Save the local audit and scan results as separate report sections.
11. Keep final HARDEN confirmation immediately before the backup and write stage.
12. Add the release tests in section 10.

Additional release controls

The new version must reject an unsupported modern protocol selection. It must not infer support from registry key existence.

The new version must flag an empty cipher result for review. It must prevent changes when the remaining usable suite set is unknown.

The current policy check covers one registry value. The release must report its detection scope, including policy sources it cannot inspect.

A failed protocol write must stop later writes. The report must retain the partial state and the recovery reference.

During the run

The order is local audit, optional baseline scan, mode selection, confirmation, backup, and local change. Then the script records AwaitingRestart.

After restart, a new run loads the previous run reference. It repeats the local audit and the same network tests.

A scan immediately after registry writes is not final verification. Existing services can retain their previous TLS state.

No automatic scheduled task is necessary for the first extension. The operator starts the verification run after restart.

8 Set up sslscan

Install and check the scanner

1. Open the official rbsec/sslscan releases page in reference R7.
2. Select a Windows binary that your organisation approves.
3. Extract the complete release to C:\Tools\sslscan.

4. Record the release version, source URL, and approved file hash.
5. Use the normal application approval process if execution is blocked.
6. Run the checks below.

```
$sslscanPath = 'C:\Tools\sslscan\sslscan.exe'  
Get-FileHash -LiteralPath $sslscanPath -Algorithm SHA256  
& $sslscanPath --version  
& $sslscanPath --help
```

The upstream project describes its Windows build as experimental. Confirm its operation on a test service before production use. [R6]

sslscan remains an executable even when PowerShell starts it. PowerShell cannot remove that dependency. An approved scan computer is an alternative.

Define the scan scope

1. Record each permitted destination IP, port, service type, and SNI name.
2. Record the application owner and TLS termination point.
3. Record the scan computer and its network location.
4. Add separate entries for public and internal service paths.
5. Include relevant IPv4 and IPv6 addresses.
6. Confirm permission and the scan period for each destination.

A target means one IP, port, SNI name, and service mode. The target list must contain every endpoint in the agreed scope.

Service example	Port example	Extra mode argument
HTTPS or direct TLS	443 or 636	None
SMTP with STARTTLS	25 or 587	--starttls-smtp
LDAP with STARTTLS	389	--starttls-ldap
RDP	3389	--rdp

The port does not select the protocol mode automatically. Use the service configuration to select the mode. [R8]

External coverage limit

A scan from the server measures its own network path. An Internet test needs a scan computer outside the internal network.

A public proxy can have different settings from the backend. Scan both paths when both are in scope. Do not expose internal services for testing.

9 Add the scan stage

PowerShell invocation example

This example runs one baseline scan. It is a development example, not a patch to version 1.0.1. Replace the example destination before use.

```
$sslscanPath = 'C:\Tools\sslscan\sslscan.exe'
$targetIp = '192.0.2.10'      # Replace with an approved IP.
$sniName = 'app.example.com' # Replace with the service name.
$targetPort = 443
$runId = [guid]::NewGuid().ToString('N')
$outputDir = Join-Path 'C:\Admin\TLS-Evidence' $runId
New-Item -ItemType Directory -Path $outputDir | Out-Null
if (-not (Test-Path -LiteralPath $sslscanPath -PathType Leaf)) {
    throw 'The sslscan program is absent.'
}
$xmlPath = Join-Path $outputDir 'baseline.xml'
$textPath = Join-Path $outputDir 'baseline.txt'
$scanArgs = @(
    '--no-colour'
    '--no-heartbleed'
    '--timeout=5'
    '--connect-timeout=10'
    '--sleep=100'
    "--sni-name=$sniName"
    "--xml=$xmlPath"
    ('{0}:{1}' -f $targetIp, $targetPort)
)
& $sslscanPath @scanArgs 1> $textPath 2> "$textPath.stderr"
$scanExitCode = $LASTEXITCODE
if ($scanExitCode -ne 0) {
    throw "The scan failed. Exit code: $scanExitCode"
}
if (-not (Test-Path -LiteralPath $xmlPath -PathType Leaf)) {
    throw 'The scan did not produce XML.'
}
```

The call operator (&) starts the program. The array passes each argument separately. No Invoke-Expression or command-string evaluation is necessary.

The example preserves SNI while connecting to a specified IP. It retains XML, console output, and error output. Heartbleed probing is disabled.

The timeouts limit individual operations, not the full scan. The delay is in milliseconds. These options are defined by sslscan. [R8]

Before release, add an overall process time limit and cancellation. Use one scan at a time for the first build.

For IPv6, format the destination as [address]:port. Insert an approved service mode argument before the destination when necessary.

9 Add the scan stage continued

Validate output before the report uses it

A zero exit code and an XML file do not prove complete coverage. The parser must check the expected target and result fields.

```
$readerSettings = New-Object System.Xml.XmlReaderSettings
$readerSettings.DtdProcessing =
    [System.Xml.DtdProcessing]::Prohibit
$readerSettings.XmlResolver = $null
$reader = [System.Xml.XmlReader]::Create(
    $xmlPath, $readerSettings)
try {
    $scanDocument = New-Object System.Xml.XmlDocument
    $scanDocument.XmlResolver = $null
    $scanDocument.Load($reader)
}
finally {
    $reader.Dispose()
}
# Next: check this document against the pinned sslscan schema.
```

This code checks XML syntax without external entity resolution. It does not classify the scan as successful or secure.

Parser build steps

1. Keep output samples from the approved sslscan release.
2. Define expected fields for target identity, protocol tests, ciphers, and certificates.
3. Reject malformed output or an unexpected schema.
4. Retain the raw result when a field is absent.
5. Record accepted, rejected, failed, and untested results separately.
6. Map cipher names to the local naming scheme before comparison.
7. Encode all imported text before insertion into HTML.
8. Mark incomplete scans as Inconclusive.

sslscan XML has changed between versions. The parser must match the approved release. SSLv2 and SSLv3 protocol checks do not enumerate individual ciphers in sslscan 2. [R6]

What the report can conclude

The report can describe the tested endpoints from the recorded scan location. It cannot claim complete external coverage without a complete target inventory.

A scan timeout does not mean that TLS is disabled. A working TLS 1.2 connection is not proof that TLS 1.0 is rejected.

The scanner does not identify every outbound application dependency. It does not change a proxy, Java runtime, or application TLS setting.

Use application-specific checks for mutual TLS, database negotiation, QUIC, certificate trust, and other paths outside the validated scanner scope.

10 Accept the release and retain evidence

Test the script controls

Test	Acceptance result
R, Enter, or unexpected mode text	No TLS registry change.
H without administrator rights	Hardening stops before backup and writes.
Final text differs from HARDEN	No TLS registry change.
Cipher audit or required registry read fails	Hardening stops with a clear failure.
Cipher policy exists	Local hardening stops.
Backup cannot complete	No hardening writes occur.
One cipher name or several Name values	Each valid name becomes a separate string.
Partial write failure	Result identifies the partial state and recovery record.
Restore an originally absent value	The new value is absent after restoration.
Unsupported TLS 1.3 selection	No TLS 1.3 value is created by that selection.

Test the scan extension

1. Do a scan of a known test service with legacy TLS enabled.
2. Confirm that the result contains the expected legacy protocol.
3. Do a scan of a known test service with legacy TLS disabled.
4. Confirm a modern handshake and the expected legacy rejection.
5. Check SNI, each service mode, IPv4, and IPv6 against known results.
6. Check timeout, blocked port, missing executable, and malformed XML results.
7. Confirm that all incomplete results show Inconclusive.

8. Confirm that a scan failure cannot authorise a local change.

Validate after the production restart

Each changed legacy protocol must show the planned local state. Network tests must match the approved endpoint policy.

Application owners must check login, mail, database access, monitoring, backup, and remote administration where applicable. Outgoing connections need their own tests.

Keep the script hash, transcript, backup, target list, scan version, scan location, UTC times, and application results together.

Close the change only after the owner accepts the evidence. Record exceptions with an owner and a review date.

11 Terms and references

Technical term	Meaning in this manual
Audit	A check of configuration or evidence.
Cipher suite	A named set of cryptographic algorithms.
Hardening	A configuration change that removes selected TLS options.
Schannel	The Windows security provider for TLS and SSL.
SNI	Server Name Indication. The TLS hostname sent by the client.
STARTTLS	An application command that starts TLS on an existing connection.
TLS termination point	The system that accepts and processes a TLS connection.
Rollback	Restoration of the recorded configuration before a change.
Inconclusive	The available evidence cannot establish the expected result.

Command names, paths, identifiers, and literal UI text keep their original spelling. These items are technical terms.

Source record

Primary implementation source: Invoke-TLSHardening.ps1, version 1.0.1. Its README.md supplies the original operating instructions.

SHA256 of the inspected script:

```
84b3ac49b9b5b2ec3deac92c6d3f69066a02b07512df50a2f02d01aecc03908c
```

Reference list

R1 ASD-STE100 Issue 9, writing rules and dictionary

https://www.asd-ste100.org/assets/files/ASD-STE100_ISSUE9.pdf

R2 Microsoft, Protocols in TLS and SSL

<https://learn.microsoft.com/windows/win32/secauthn/protocols-in-tls-ssl--schannel-ssp->

R3 Microsoft, TLS registry settings

<https://learn.microsoft.com/windows-server/security/tls/tls-registry-settings>

R4 Microsoft, Get-TlsCipherSuite

<https://learn.microsoft.com/powershell/module/tls/get-tlsciphersuite>

R5 Microsoft, Disable-TlsCipherSuite

<https://learn.microsoft.com/powershell/module/tls/disable-tlsciphersuite>

R6 rbsec, sslscan README

<https://github.com/rbsec/sslscan>

R7 rbsec, sslscan releases

<https://github.com/rbsec/sslscan/releases>

R8 rbsec, sslscan command manual

<https://github.com/rbsec/sslscan/blob/master/sslscan.1>